单元测试修复 Agent · 学习文档

> 面向学习者：读完并跟着做一遍，你应该能**闭卷讲清这个项目是什么、怎么工作的、为什么这么设计**。
> 阅读对象：想把这个项目内化成"自己的东西"的你。
> 建议时长：2～3 天（每天 1～2 小时，动手为主、读代码为辅）。

---

0. 一句话定位

> **一个会根据 pytest 失败反馈、自己反复读代码、改代码、跑测试的迷你 Coding Agent。**
> 每次运行都发生在**独立的 Git worktree** 里；失败会回滚，成功会导出 `final.patch`；全过程写 JSONL 轨迹。

它的**灵魂**只有一句：**模型只出主意，程序才动手，成败由客观裁判（pytest）说了算。**

---

1. 项目整体概述与目标

1.1 它解决什么问题

给你一个"有一个测试失败"的代码仓库，它自动完成：

```
失败测试 → 读代码定位 → 精确修改 → 重新跑测试 → 根据新报错继续迭代 → 目标测试通过
                                                                    ↓
                                                          再跑【完整测试集】防回归
                                                                    ↓
                                                  通过 → 导出 final.patch（人工审查后应用）
```

1.2 设计目标（按重要性排序）

目标	含义	落到代码哪里
可验证	修没修好不由模型自说自话，由 pytest 判定	`run_tests` 返回 `passed`，目标通过后还要 `verify` 全量
可审查	改动要小、要能被人看懂	`edit_file` 精确替换，禁止整文件重写
不破坏原仓库	绝不直接改你的代码	`workspace.py` 的 git worktree 隔离
失败可回滚	修坏了要能回到原状	`rollback()`：`git reset --hard` + `clean -fd`
可追溯	每一步都能复盘	`trace.py` 写 JSONL 轨迹
有界	不会无限烧钱/死循环	`MAX_STEPS` 步数预算

1.3 和"让 LLM 一次性生成答案"的本质区别

普通做法：问模型一句 → 它吐一段代码 → 完事（对不对没人管）。
本项目：模型每次只能**发起一个工具调用**（读/搜/改/测），拿到**真实的工具结果**后再决定下一步，形成一个"观察 → 决策 → 行动 → 再观察"的循环。**反馈来自真实执行环境，不是模型想象。**

---

2. 核心功能清单

功能	说明
有界 Agent 循环	LangGraph 状态图：`decide → approve → act → verify → finish`
四个真实工具	`read_file`（带行号）/ `edit_file`（精确替换）/ `search_code` / `run_tests`
模型接入	LangChain 的 OpenAI 兼容 Function Calling（默认 DeepSeek，可换）
worktree 隔离	原仓库不被直接修改，改动发生在 detached worktree
HITL 人工确认	修改生产文件前可暂停，由你批准/拒绝
SQLite checkpoint	中断或暂停后可恢复，不会丢状态
全量回归验证	目标测试通过后，自动跑 `pytest .` 检查是否引入新问题
失败回滚 + 补丁分离	失败清空 worktree；`attempt.patch`（复盘用）与 `final.patch`（成功）分开
轨迹可观测	JSONL 记录模型决策、工具参数、测试观察、token、最终状态
基准与评测	20 个人为注入的缺陷，统计成功率/平均步数/token/成本
MCP 外部工具	可配置 stdio/sse 服务器，工具以 `mcp__*` 形式交给模型

---

3. 目录结构与模块说明

```
unit-test-repair-agent/
├── repair_agent/              # 核心包（14 个 .py）
│   ├── agent.py               # 运行生命周期、基线、patch、暂停/恢复、结果汇总
│   ├── graph.py               # LangGraph 节点与条件边：decide/approve/act/verify/finish
│   ├── models.py              # 模型协议、LangChain 工具调用适配器
│   ├── mcp.py                 # MCP 工具接入：stdio/sse + mcp__* 调度
│   ├── tools.py               # 四种工具、schema、路径防护、pytest 执行
│   ├── workspace.py           # git worktree、diff、rollback、cleanup
│   ├── benchmark.py           # 缺陷任务加载、临时仓库生成、基准有效性检查
│   ├── evaluation.py          # 顺序评测、指标聚合、报告输出
│   ├── prompts.py             # 系统提示词与初始失败反馈
│   ├── config.py              # .env 配置与参数校验
│   ├── trace.py               # JSONL trace 与 JSON 落盘
│   ├── cli.py                 # run / resume / evaluate / validate-benchmark
│   └── __main__.py            # python -m repair_agent 入口
├── benchmarks/
│   ├── base_project/          # 原始 numberkit 小包 + 17 个 pytest 测试
```

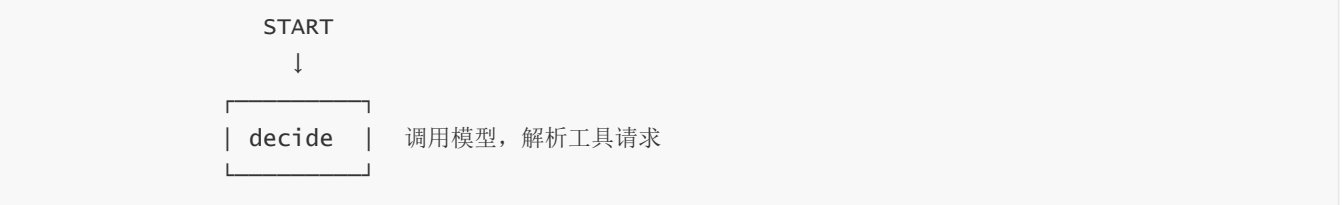
```
|   └── mutations.json        # 20 个代码变异（人为缺陷）
├── tests/                    # 5 个测试文件（离线假模型，不调 API）
├── examples/
|   └── demo_mcp_server.py    # 最小 stdio MCP 服务器（演示 MCP 接入）
├── .env.example
├── pyproject.toml
└── runs/                     # 运行后生成，已 gitignore
```

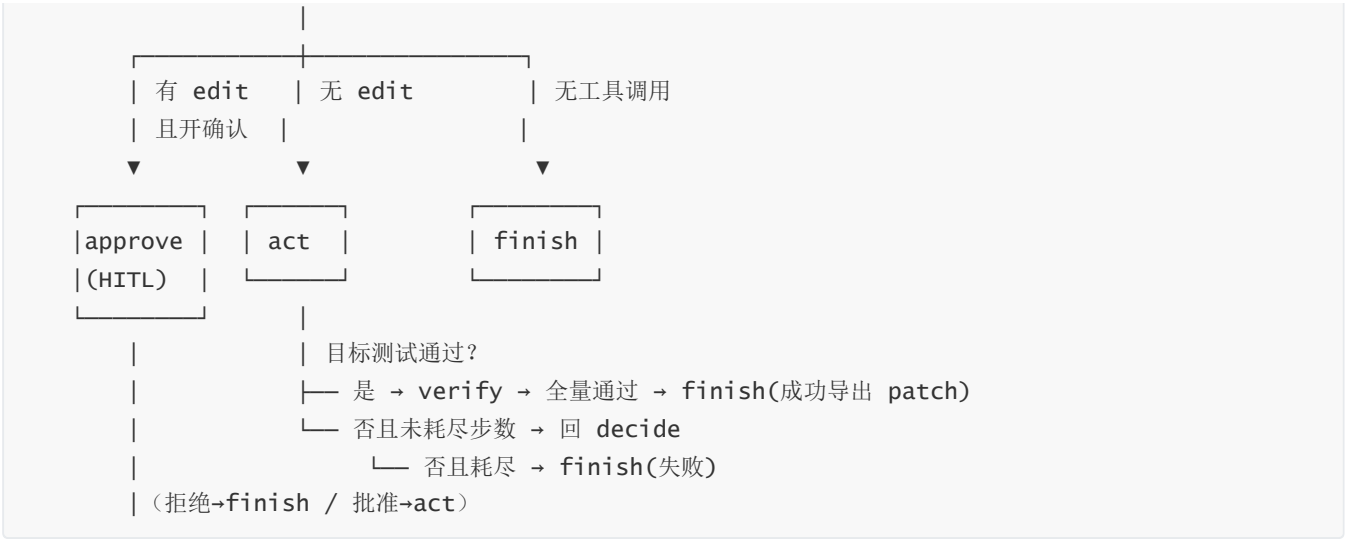
每个模块一句话 + 关键符号

模块	一句话职责	关键类/函数
`cli.py`	解析命令行，分发到 run / resume / evaluate / validate-benchmark	`main()`
`config.py`	从 `.env` 读配置并校验	`Settings.from_env()`
`models.py`	统一模型接入，把模型返回转成内部 `ToolCall`	`LangChainModel.complete()`
`prompts.py`	系统提示词 + 初始失败反馈	`SYSTEM_PROMPT`, `initial_user_prompt()`
`graph.py`	定义状态图节点与路由	`build_repair_graph()`, `decide/act/verify`
`agent.py`	生命周期编排（创建 worktree、跑图、收尾）	`RepairAgent.run()/resume()`
`tools.py`	四个工具 + 路径防护 + pytest 执行	`ToolBox.execute()`, `TOOL_SCHEMAS`
`workspace.py`	git worktree、diff、rollback、cleanup	`GitWorktree`
`mcp.py`	MCP 服务器接入与工具调度	`MCPToolRegistry`
`benchmark.py`	加载 20 个缺陷、生成临时仓库、校验有效性	—
`evaluation.py`	顺序跑评测、聚合指标、写报告	—
`trace.py`	JSONL 轨迹 + JSON 落盘	`JsonlTrace`

4. 关键代码结构与设计思路解析

4.1 主链路（状态图）





关键节点职责（`graph.py`）：

节点	做什么	关键逻辑
`decide`	调模型，累加步数/token，解析 `tool_calls`	无工具调用 → 记 `reason=model_stopped_without_passing_tests`
`approve`	有 `edit_file` 且开确认时，`interrupt()` 暂停等人工	拒绝 → `reason=edit_rejected_by_user`
`act`	逐个执行工具，结果写成 `tool` 消息	`run_tests` 的 `passed` 决定 `target_passed`
`verify`	目标通过后跑 `pytest .`	全量失败 → 新观察回喂 `decide`
`finish`	归一化最终 status/reason	只归一化，不做破坏性操作

4.2 状态（`RepairState`）

```python
class RepairState(TypedDict, total=False):
    messages: list          # 完整对话（system/user/assistant/tool）
    pending_calls: list     # 本轮待执行的工具调用
    steps: int              # 已用模型决策步数
    tokens: dict            # 累计 token
    target_passed: bool     # 目标测试是否已通过
    status: str             # success / failed / paused
    reason: str             # 结束原因（all_tests_passed / step_budget_exhausted / ...）
```

4.3 八个设计取舍（面试必考，逐个能讲）

1. **为什么用 LangGraph 而不是手写 while 循环**：条件分支、checkpoint、`interrupt` 人工中断散落在多个 if 里会失控；LangGraph 把它们声明成显式节点和条件边，可持久化、可恢复。
2. **为什么生命周期放 `agent.py` 而不是塞进图**：worktree/Git 进程等不可序列化对象在 `agent.py` 处理，图里只维护可序列化状态（消息/步数/token），暂停/恢复才可靠。

3. **为什么用 LangChain**：统一模型调用、消息结构、工具绑定，把返回转成内部 `ToolCall`，换供应商只改 `.env`。
4. **为什么 MCP 工具加 `mcp__<server>__<tool>` 前缀**：避免与内置工具重名，trace 一眼能看出是外部工具；MCP 是附加能力，连不上就记进 trace 继续用内置工具，不影响主闭环。
5. **为什么 `edit_file` 用精确替换**：整文件重写易引入无关改动、patch 难审。精确替换要求 `old_string` 唯一（0 次或多次都报错回喂模型）。代价是必须先读文件——这是**有意的可审计性**。
6. **为什么原仓库始终不变**：所有编辑在 detached worktree；失败 `reset --hard` + `clean -fd`，作用范围仅本次 worktree。
7. **为什么目标通过还不够**：只修一个测试可能破坏别的；所以 `verify` 默认跑全量 `pytest .`。
8. **为什么先校验缺陷**：`validate-benchmark` 要求 CLEAN 必须 pass、MUTANT 必须 fail，避免"注入后测试仍通过"的无效题污染成功率。

4.4 安全边界（`tools.py` 里很关键）

- 路径防护 `_path()`：解析后必须仍在 worktree 内，否则报 "Path escapes the isolated workspace"；`.git` 禁止访问。
- `edit_file` 拒绝改 `tests/`、`test/`、`test_*.py`——**防止模型通过改测试伪造成功**。
- `run_tests` 不用 `shell=True`，只接受 pytest 目标，拒绝 pytest flags（`-` 开头）。

---

5. 主要依赖与技术栈

来自 `pyproject.toml`：

依赖	用途	为什么
`langchain` + `langchain-openai`	模型接入与工具绑定	统一 OpenAI 兼容接口，不绑死某 SDK
`langgraph`	状态图编排	显式节点/条件边/checkpoint/interrupt
`langgraph-checkpoint-sqlite`	状态持久化	暂停后可恢复
`python-dotenv`	读 `.env`	密钥与配置分离
`mcp`	外部工具接入	stdio/sse 服务器
`pytest`（dev）	裁判 + 本项目自测	客观判定 + 回归

运行时还依赖系统里的 **Git**（`git` 命令必须可用）。

---

6. 典型使用流程（操作示例）

6.1 环境准备

```
cd d:\code\unit-test-repair-agent
# 用 uv（推荐）
uv venv --python 3.13
uv pip install --python .venv/Scripts/python.exe -e ".[dev]"
# 或普通 pip
py -3 -m venv .venv
.venv\Scripts\python.exe -m pip install -e ".[dev]"
```

6.2 第一步：零成本验证（不花 API 费）

```
.venv\Scripts\python.exe -m pytest                # 本项目自测（假模型，不调网络）
.venv\Scripts\python.exe -m repair_agent validate-benchmark
# 预期：CLEAN expected=pass / M01 expected=fail / ... / Validated 20 mutations
```

第二条做两件事：① 原始 numberkit 全量测试必须通过；② 每个变异版本必须至少让一个测试失败。**这两步不过，别急着配模型。**

6.3 配置真实模型

```
Copy-Item .env.example .env
```

编辑 `.env`：

```
LLM_API_KEY=你的密钥
LLM_BASE_URL=https://api.deepseek.com
MODEL_ID=deepseek-chat
```

> 模型必须支持 Function Calling / tool calls。纯推理模型跑不了本项目。

6.4 只跑一个缺陷任务（先 M01，别一上来 20 题）

```
.venv\Scripts\python.exe -m repair_agent evaluate --task M01 --max-steps 8
```

产物在 `runs/evaluation_时间戳/`，重点看：

```
runs/evaluation_时间戳/
├── evaluation.md / evaluation.json    # 指标报告
└── tasks/M01/
    ├── trace.jsonl                    # 每一步决策与工具观察
    ├── baseline.json                  # 修复前失败输出
    ├── checkpoints.sqlite             # LangGraph 状态检查点
    ├── attempt.patch                  # 最后一次尝试（无论成败）
    ├── final.patch                    # 仅成功时保留
    └── summary.json                   # 状态/步数/token/耗时/回滚
```

如何确认真的工作了：`summary.json` 里 `success=true`、`reason=all_tests_passed`；`trace.jsonl` 有 `edit_file` / `run_tests` / `full_suite_verification`；`final.patch` 非空。

6.5 运行完整 20 题评测

```
.venv\Scripts\python.exe -m repair_agent evaluate --max-steps 12
.venv\Scripts\python.exe -m repair_agent evaluate --limit 3        # 调试期只跑前 3 题
.venv\Scripts\python.exe -m repair_agent evaluate --task M03 --task M07
```

6.6 修复自己的 Git 仓库

前提：已 init、有至少一个 commit、无未提交的已跟踪改动、测试依赖已装进本项目环境。

```
.venv\Scripts\python.exe -m repair_agent run `
  --repo d:\path\to\your-repo `
  --test tests/test_example.py::test_case `
  --max-steps 12
```

成功后**不会直接改你的仓库**，而是生成 `final.patch`，人工审查后：

```
cd d:\path\to\your-repo
git apply --check <runs目录>\final.patch
git apply <runs目录>\final.patch
```

6.7 HITL 人工确认 + 断点恢复

```
.venv\Scripts\python.exe -m repair_agent run --repo ... --test ... --approve-edits
# 终端会打印将要改的文件/旧串/新串，输入 y 批准 / n 拒绝回滚 / q 退出保留现场
# 之后恢复：
.venv\Scripts\python.exe -m repair_agent resume --run-dir runs\你的运行目录
```

`resume` 读 `run_context.json`、重连 worktree、用原 `thread_id` 从 SQLite checkpoint 恢复。评测模式 `evaluate` 总是关闭人工确认以保证可复现。

---

7. 常见问题与注意事项

问题	原因 / 处理
`Missing LLM_API_KEY`	没建 `.env` 或没填 Key；离线测试不需要，`run` / `evaluate` 需要
模型返回文本但没工具调用	模型不支持 Function Calling；看 `trace.jsonl` 确认实际响应
目标仓库测试无法启动	本项目用当前 `.venv` 的 Python；把目标仓库测试依赖也装进该环境

问题	原因 / 处理
`Target repository has tracked uncommitted changes`	先 commit 或 stash；程序故意不在状态不明的仓库建基线
pytest 超时	增大 `TEST_TIMEOUT_SECONDS`，或缩小测试目标
中文乱码	子进程已设 `PYTHONUTF8=1`；终端乱码可 `$env:PYTHONUTF8="1"` 后重试
成本估算为空	预期行为；只有配置非零 token 单价才估算
`resume` 提示 worktree 缺失	别手动删 `runs/.../worktree` 和 `checkpoints.sqlite`
恢复时换了模型/Key	换模型后继续旧 checkpoint，结果不能和原运行混着统计

注意事项（红线级）：

- 只支持修改**已有文本文件**，没有"新建文件"工具；
- 不是通用 shell Agent，也**不是安全的多租户沙箱**，请在可丢弃的测试仓库里用；
- 内置 20 题是**教学基准**，不代表真实大型项目修复能力（面试别夸大）；
- 任何成功率数字**必须自己 `evaluate` 跑出来**，换模型后数字会变，不能混用。

---

8. 阅读顺序建议（按"能讲清"的目标分层）

第 1 层 · 先跑起来（建立整体印象）

1. 安装 → `pytest` → `validate-benchmark`（零成本验证闭环）
2. 配模型跑一次 `evaluate --task M01`，打开 `trace.jsonl` 从头读到尾，**对照状态图**理解每一步

第 2 层 · 读懂编排（这是项目的心脏）

3. `prompts.py`（先看模型被要求怎么思考）
4. `graph.py`（decide/act/verify 三个节点 + 四个路由函数，全文约 220 行）
5. `agent.py`（看生命周期怎么包住图：worktree → 跑图 → 收尾）

第 3 层 · 读懂工具与安全（最能体现"真的懂"）

6. `tools.py`（重点看 `_path()` 路径防护、`edit_file` 精确替换、`run_tests` 不用 shell）
7. `workspace.py`（worktree 创建 / diff / rollback / cleanup，全文不到 80 行）

第 4 层 · 读懂工程件

8. `models.py`（LangChain 怎么接、token 怎么统计）
9. `mcp.py`（MCP 怎么连、为什么加前缀）+ `examples/demo_mcp_server.py`
10. `benchmark.py` + `evaluation.py`（评测怎么保证干净、指标怎么算）

第 5 层 · 闭卷验收

11. 不看文档画出状态图，说出每个 `reason` 在什么情况下产生

12. 回答三个追问：为什么 worktree 而不是直接改？为什么精确替换而不是重写？为什么目标通过还要跑全量？

---

9. 自检清单（能讲 / 能举证 / 能改）

能讲

- [ ] 一句话说清项目在解决什么问题
- [ ] 闭卷画出状态图，说出五个节点职责
- [ ] 说出 8 个设计取舍里的至少 5 个"为什么"

能举证

- [ ] 打开一份 `trace.jsonl`，讲清某一步"模型决定做什么 → 工具返回什么"
- [ ] 拿出自己 `evaluate` 跑出的成功率（不是抄 README 的）
- [ ] 说清 `final.patch` 和 `attempt.patch` 的区别及各自用途

能改

- [ ] 改 `MAX_STEPS` 并解释成功率/步数会怎么变（可对照 RESULTS.md 的消融）
- [ ] 关掉 `VERIFY_ALL_TESTS`，说出这样做的风险

扛追问

- [ ] 「真是你写的吗？」→ 如实说灵感来源 + 砍掉了什么 + 为什么这么设计
- [ ] 「MCP 有什么价值？」→ 说清工具标准化/跨端复用，以及为什么单项目里感受不明显
- [ ] 「20 题成功率能代表什么？」→ 明确说这是教学基准，不代表真实项目